

# ICARUS-LEO

## Simulator Architecture

In-network caching over LEO satellite constellations  
with channel losses, handover and chunk-level delivery

G. Campobello

Department of Engineering, University of Messina, Italy

Document version 1.0, September 2026. Describes ICARUS-LEO release 2.42.

This document is licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0),  
<https://creativecommons.org/licenses/by/4.0/>.

## Contents

|          |                                                |          |
|----------|------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                            | <b>2</b> |
| <b>2</b> | <b>The ICARUS framework</b>                    | <b>2</b> |
| <b>3</b> | <b>Architecture overview</b>                   | <b>3</b> |
| <b>4</b> | <b>LEO network model</b>                       | <b>4</b> |
| 4.1      | Nodes and links . . . . .                      | 4        |
| 4.2      | Constellations . . . . .                       | 4        |
| 4.3      | Routing . . . . .                              | 5        |
| 4.4      | Static and dynamic topologies . . . . .        | 5        |
| <b>5</b> | <b>Handover dynamics</b>                       | <b>6</b> |
| <b>6</b> | <b>Loss modelling</b>                          | <b>6</b> |
| <b>7</b> | <b>Chunk-level delivery and retransmission</b> | <b>7</b> |
| 7.1      | From content objects to chunks . . . . .       | 7        |
| 7.2      | Sessions and retransmission . . . . .          | 7        |
| 7.3      | Timing . . . . .                               | 8        |
| <b>8</b> | <b>Result collectors</b>                       | <b>8</b> |
| <b>9</b> | <b>Modelling assumptions</b>                   | <b>9</b> |

# 1 Introduction

ICARUS-LEO is a simulator for the evaluation of in-network caching in Information-Centric Networking (ICN) deployed over Low Earth Orbit (LEO) satellite constellations. It extends ICARUS [1], a widely used flow-level simulator for ICN caching, with the elements that distinguish a LEO network from the wired networks ICARUS was designed for: lossy communication links, time-varying connectivity caused by orbital motion, satellite handovers, and the delivery of contents as sequences of chunks that can be lost and retransmitted individually.

The reference scenario is a content delivery network in which user terminals retrieve contents stored on terrestrial origin servers through a constellation of cache-enabled satellites. A request travels from the user terminal to its serving satellite, across the inter-satellite links and, if no cache on the path holds the requested chunk, down to a ground station and to the origin server; the data follows the reverse path.

This document describes what the simulator does and how it is organised, at the level of its architecture. It is intended for readers who wish to understand the modelling choices of the simulator, or who plan to use it for their own studies. It does not describe how to prepare a configuration or how to read the output files, which are covered by the user guide of the repository, and it does not report simulation results.

The remainder of the document is organised as follows. Section 2 summarises the ICARUS framework on which the simulator is built. Section 3 gives an overview of the architecture. Sections 4 to 8 describe the LEO network model, the handover model, the loss models, the chunk-level delivery process and the result collectors. Section 9 summarises the main modelling assumptions.

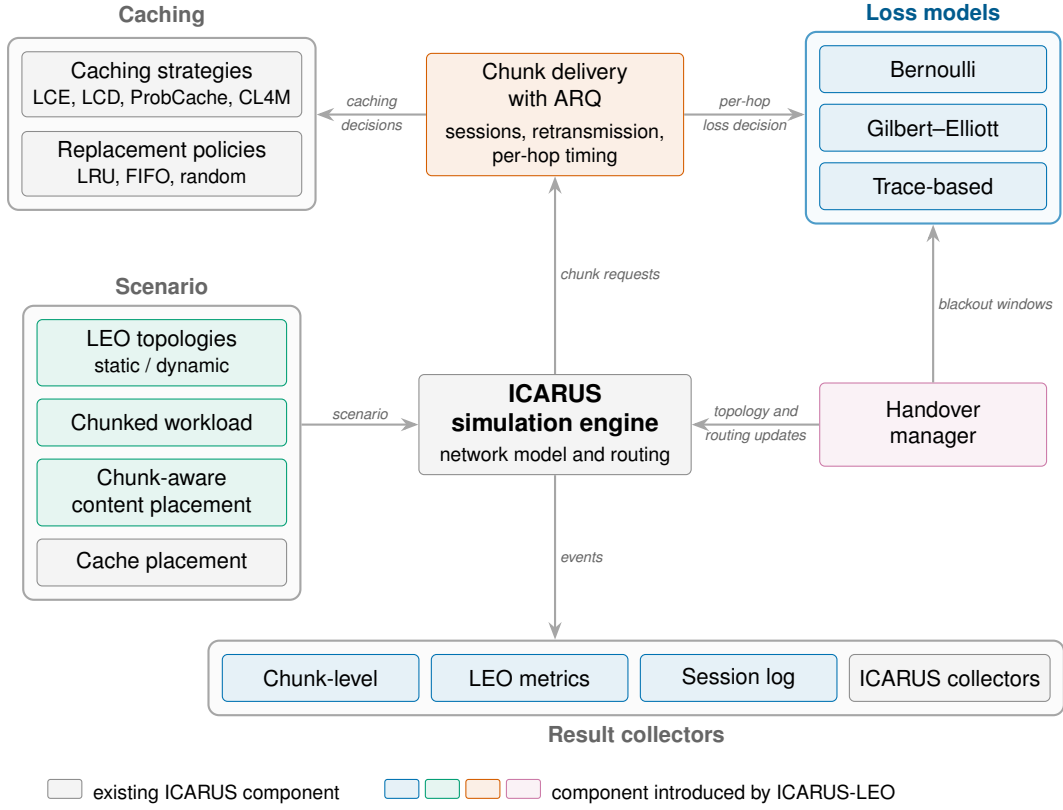
## 2 The ICARUS framework

ICARUS is a Python simulator for ICN caching that operates at the flow level. A simulation is described by a configuration that defines a queue of experiments; each experiment combines a set of interchangeable components, each selected by name from a registry to which new implementations can be added through a plug-in mechanism:

- a *topology*, i.e. a graph whose nodes are routers, receivers and sources;
- a *content placement*, which assigns each content of the catalogue to an origin server;
- a *cache placement* and a *cache replacement policy*, such as LRU, FIFO or random replacement;
- a *workload*, which generates a sequence of requests, each identified by a time, a receiver and a content, with Zipf-distributed content popularity [2];
- a *caching strategy*, which decides how each request is forwarded and where the returning content is cached, for example Leave Copy Everywhere (LCE), Leave Copy Down (LCD) [3], ProbCache [4] and Cache Less for More (CL4M) [5];
- one or more *data collectors*, which observe the simulation and compute performance metrics such as the cache hit ratio, the latency and the link load.

The simulation engine replays the workload request by request: for each request, the strategy moves the request and the corresponding content through the network, and the collectors are notified of every relevant event.

ICARUS makes a set of assumptions that suit wired networks but not LEO constellations. The topology is a static graph, links never lose packets, and content is handled at the level of the *content object*: each request retrieves a whole object as a single, indivisible transfer, and only the propagation delay is accounted for. Consequently, ICARUS cannot represent partial losses, retransmissions, the transmission time of packets on low-capacity radio links, or the effect of satellite motion on the path followed by requests.



**Figure 1:** Architecture of ICARUS-LEO. Grey blocks are components of the original ICARUS framework; coloured blocks are introduced by ICARUS-LEO. The engine hands each chunk request to the delivery process, which forwards requests and data hop by hop, applies the loss model of each link, asks the caching strategy where to store copies and, when the request or the data is lost, retransmits the request. The handover manager keeps the topology and the routing aligned with the simulation time and provides the handover blackout windows to the loss models. The engine feeds all result collectors.

### 3 Architecture overview

ICARUS-LEO extends the caching model of ICARUS with four groups of functions:

1. **LEO topologies with handover dynamics:** standard Walker constellations, static or time-evolving, in which user terminals periodically re-associate to a new serving satellite (Sections 4 and 5);
2. **a per-link loss modelling layer**, ranging from synthetic models to the replay of measured loss traces (Section 6);
3. **chunk-level content delivery with retransmissions**, in which every content is split into chunks that are requested, transferred, cached and, if lost, retransmitted individually (Section 7);
4. **dedicated result collectors** for chunk-level and satellite-network metrics (Section 8).

Figure 1 shows how these functions relate to the components of ICARUS. All extensions are *additive*: they are separate modules registered through the native plug-in mechanism of ICARUS, and the core simulation logic is not modified. Experiments that do not use the new components run exactly as in the original simulator.

ICARUS-LEO does not alter the caching logic. The chunk delivery process decides how requests and data travel through the lossy network, whereas the caching decision taken at each node reached by the data is that of the original ICARUS strategy (LCE, LCD, ProbCache or CL4M, the strategies for which a loss-aware delivery is provided), and the cache contents are managed by the original replacement policies. On a static topology with no loss model and single-chunk contents, requests and data follow the same

paths and the strategies take the same caching decisions as in the original simulator; the only difference is that latencies also include the transmission times of the packets.

**Execution flow.** For each experiment, the scenario is built first: the constellation and its ground segment, the assignment of contents to origin servers, the placement of caches on the satellites and the sequence of content requests. The engine then replays the requests. Each content request is expanded into one request per chunk, and each chunk request is handled as a *session*. At the start of a session, the handover manager brings the topology to the current simulation time, updating the serving satellites, the ground links and the routing accordingly. The request is then forwarded hop by hop along the shortest path towards the origin server; at every hop the loss layer decides whether the packet is delivered or lost, and every on-path cache is looked up. The returning data is subject to the same per-hop loss decisions, and the caching strategy decides where copies are stored. If the request or the data is lost, the receiver retransmits the request after a timeout, up to a configurable number of attempts. The collectors observe every hop, cache hit, loss and retransmission, and compute the metrics at the end of the experiment.

## 4 LEO network model

### 4.1 Nodes and links

A constellation is modelled as a graph with four types of nodes (Figure 2):

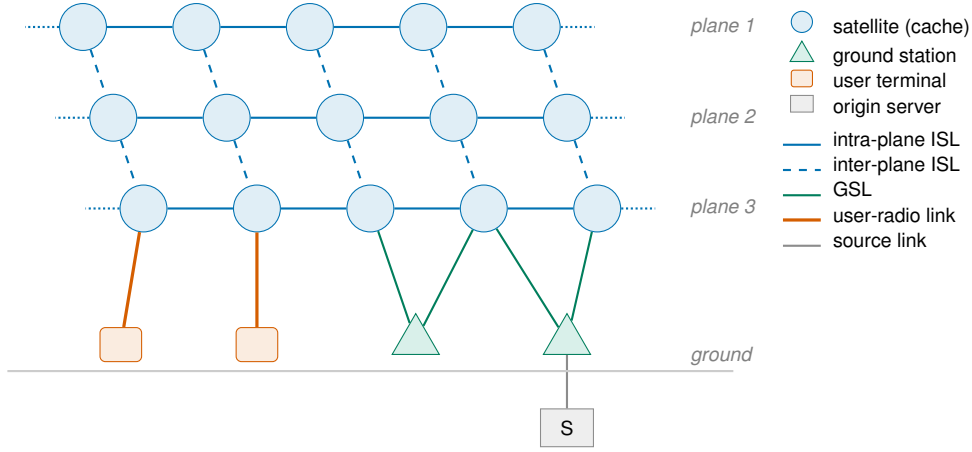
- **satellites**, which forward requests and data and are the cache-enabled nodes of the network;
- **ground stations**, which act as gateways between the satellite segment and the terrestrial infrastructure and do not cache content;
- **user terminals** (receivers), which issue the requests and communicate directly with their serving satellite over a radio link;
- **origin servers** (sources), which permanently store the contents and are attached to ground stations.

The nodes are connected by five types of links: intra-plane inter-satellite links (ISL), which connect consecutive satellites of the same orbital plane; inter-plane ISLs, which connect satellites of adjacent planes; ground-satellite links (GSL) between ground stations and satellites; user-radio links between user terminals and their serving satellite; and terrestrial source links between origin servers and ground stations. The link type determines the loss model applied to the link (Section 6) and its transmission capacity, which in turn sets the transmission time of each packet. The propagation delay of the satellite links (ISL, GSL and user-radio links) is derived from the geometric distance between their endpoints, so that latency reflects the actual spatial configuration of the constellation; terrestrial source links have a small, fixed delay.

### 4.2 Constellations

The graph is generated from a small set of orbital parameters: number of planes, satellites per plane, altitude, inclination and inter-plane phasing. Satellites follow ideal circular orbits, and their positions are computed analytically. Two classical Walker patterns [6] are supported, which differ in how the orbital planes are spread in right ascension:

- in a **Walker-Delta** constellation (e.g. Starlink) the planes are spread over the full  $360^\circ$ , which suits the moderate inclinations used for global coverage; all planes are co-rotating and inter-plane ISLs connect every pair of adjacent planes, including the last and the first;
- in a **Walker-Star** constellation (e.g. Iridium) the planes are spread over  $180^\circ$ , giving a near-polar, pole-to-pole geometry in which adjacent planes converge at the poles. The first and last planes are



**Figure 2:** Nodes and link types of the LEO network model (three planes and a few satellites shown). Each ground station keeps links to two visible satellites; each user terminal is served by one satellite at a time.

counter-rotating and form a *seam*, across which no inter-plane ISL is established; inter-plane ISLs are also deactivated at high latitudes, where adjacent planes come too close to be tracked.

In both patterns, an inter-plane ISL exists only while its two satellites are in line of sight of each other. Two representative constellations are provided as presets (Table 1); custom constellations can be defined from the same parameters.

**Table 1:** Constellation presets.

| Preset        | Pattern      | Planes $\times$ satellites | Altitude | Inclination  |
|---------------|--------------|----------------------------|----------|--------------|
| Iridium-like  | Walker-Star  | $6 \times 11 = 66$         | 781 km   | $86.4^\circ$ |
| Starlink-mini | Walker-Delta | $12 \times 11 = 132$       | 550 km   | $53^\circ$   |

Ground stations are placed at configurable geographical locations, by default at major metropolitan areas worldwide, and each one is connected to the closest visible satellites above a minimum elevation angle, two by default. User terminals can either be co-located with the ground stations, modelling a regional access scenario, or be distributed over the service area, modelling direct-to-device access; in the latter case their positions are chosen so that every terminal sees at least one satellite at each topology snapshot. Origin servers are attached to ground stations spread across the network.

### 4.3 Routing

Requests are forwarded along minimum-hop paths in the current topology, and the paths are updated whenever the topology changes. In dynamic topologies, a configurable option determines whether ground stations may relay transit traffic between satellites: when relaying is disabled, a ground station is used only as the gateway to the origin servers, and satellite-to-satellite traffic stays on the inter-satellite links whenever an inter-satellite path exists.

### 4.4 Static and dynamic topologies

Topologies are available in two forms. A **static** topology is a single snapshot of the constellation; it isolates the effect of the channel on caching, in the absence of mobility. In a **dynamic** topology, connectivity evolves over time as the satellites move along their orbits: user terminals re-associate to a new serving satellite, ground-station links are reassigned to the currently visible satellites, the lengths and propagation delays of the inter-satellite links are updated, and inter-plane links are activated or deactivated as the

geometry changes. Each change of serving satellite alters the forwarding path and, consequently, the set of caches encountered by the following requests.

## 5 Handover dynamics

A dedicated *handover manager* coordinates the dynamic topology with the rest of the simulator. At the start of each session it advances the constellation to the current simulation time, applies any change of serving satellite, keeps the routing of the engine consistent with the new topology, and makes the handover blackout windows available to the loss layer. On static topologies it is transparent.

Two handover models are available.

**Analytical model (default).** A satellite is visible from a terminal while it remains above a configurable minimum elevation angle. A terminal keeps its serving satellite until it sets below that angle and then selects the visible satellite with the highest elevation. The handover instants of each terminal are computed from the orbital geometry, independently of the topology snapshots, and the user-radio link is rewired accordingly. The model distinguishes two kinds of interruption:

- a short *resynchronisation* window, during which the terminal re-associates to a satellite that is already visible; its duration is set by the user, either as a fixed value or as a draw from a uniform distribution;
- a longer *coverage-gap* window, which arises only when no satellite is visible; it spans the interval without visible satellites, determined by the geometry, followed by the acquisition of the next satellite.

Ground stations keep two simultaneous links to distinct satellites and follow a make-before-break policy. A change of satellite therefore causes no loss at a ground station; a ground station is blacked out only during a coverage gap, when no satellite at all is visible from it, and the blackout then spans the gap and the acquisition of the next satellite.

**Snapshot model.** The topology is sampled at a sequence of discrete instants, and a blackout of configurable duration is applied to the user-radio link at every snapshot at which the serving satellite changes, and to every ground-station link that is reassigned to a different satellite. The model is less faithful but can be cheaper when snapshots are sparse, and its behaviour approaches that of the analytical model as the snapshot density increases.

In both models the re-association always takes place, so the topological consequences of handover on caching are captured regardless of the loss it induces. A packet whose transmission on an affected link overlaps a blackout window is lost, independently of the channel loss model in use.

## 6 Loss modelling

The loss layer decides, for every transmission of a packet over a link, whether the packet is delivered or lost. The decision is taken hop by hop, for requests and data alike, and depends on the link type; for the models based on bit-error rates it also depends on the size of the packet, so that a data chunk, much larger than a request, is more exposed to bit errors. All loss models share a common interface, so that additional models can be integrated through the plug-in mechanism.

Each link type can be associated with its own loss model and parameters, so that different portions of the network exhibit different channel characteristics. For example, a measured trace can drive the user-radio links while a Bernoulli model is applied to the ground-satellite and inter-satellite links and the terrestrial links are kept lossless. The following models are available, in increasing order of realism:

- **Bernoulli.** Memoryless losses: a packet of  $L$  bits is lost with probability  $1 - (1 - \beta)^L$ , where  $\beta$  is the bit-error rate configured for the link type; alternatively, a packet loss probability can be set directly.
- **Gilbert–Elliott.** A two-state (good/bad) Markov chain [7, 8], maintained independently for each link and advancing with every packet transmitted on it, reproduces bursty losses that are correlated over consecutive transmissions; each state has its own bit-error rate, which can differ between link types.
- **Trace-based.** A pre-recorded binary loss trace is replayed: a packet is dropped wherever the corresponding trace entry indicates a loss. The trace can be measured on a real LEO link or generated synthetically by another model. The trace is read circularly and the position along it advances with each transmission; the same trace can drive several links at once, each reading it from an independent, random offset.

A lossless configuration is also available and reproduces the ideal links of the original ICARUS.

Channel losses and handover losses are combined: a packet is first checked against the handover blackout windows of the link (Section 5) and, if it is not lost to a blackout, it is submitted to the channel model of the link.

## 7 Chunk-level delivery and retransmission

### 7.1 From content objects to chunks

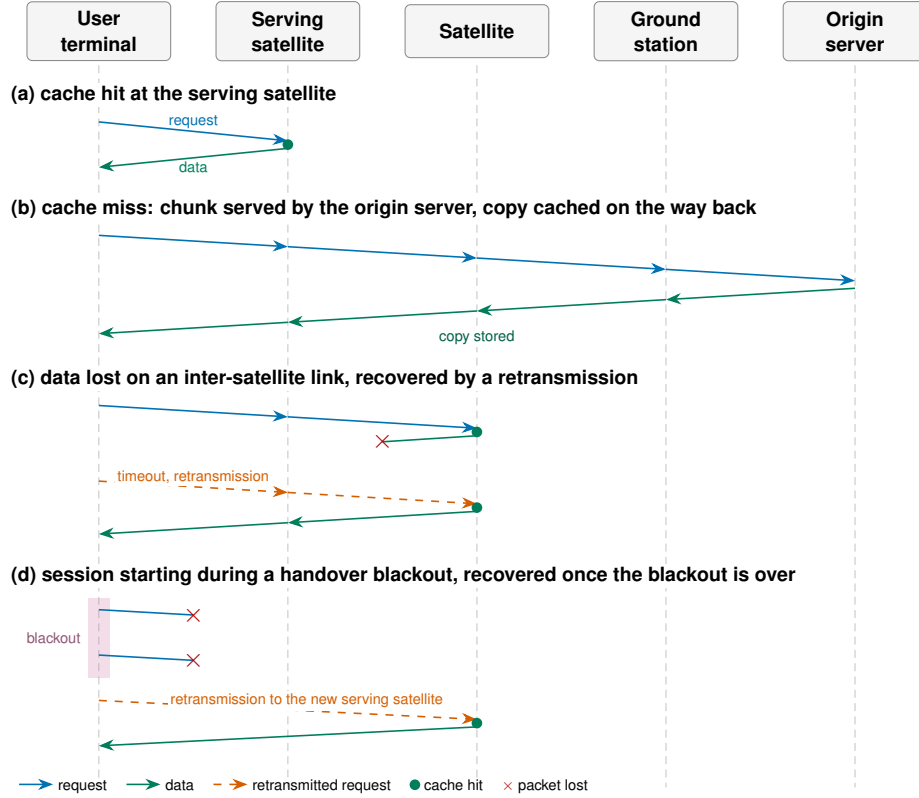
ICARUS-LEO replaces the content object of ICARUS with the *chunk* as the unit of transfer and caching. Content requests follow a stationary workload in which popularity is Zipf-distributed over a configurable catalogue. Each content is split into a fixed number  $N_c$  of chunks, and each content request generates  $N_c$  chunk requests, issued together by the same terminal; each chunk is carried by a single data packet. A chunk-aware content placement ensures that all the chunks of a content are stored on the same origin server. With  $N_c = 1$  the model reduces to the object-level model of ICARUS.

A content is *delivered* only when all its chunks have been received. This finer granularity allows the simulator to capture partial losses, retransmissions and incomplete deliveries, and to evaluate performance both at the chunk level and at the content level.

### 7.2 Sessions and retransmission

The delivery of a single chunk, including any retransmission, is called a *session* (Figure 3). The request travels from the terminal towards the origin server and is answered by the first node on the path that holds the chunk, either an on-path cache or the origin server. The data travels back along the reverse path, and the caching strategy decides at which nodes a copy is stored. A packet that is lost does not travel further: nodes beyond the loss point neither see the request nor receive the data.

Since the original ICARUS assumes lossless links, it provides no recovery mechanism. ICARUS-LEO introduces an ARQ-style scheme driven by the receiver: if the data has not arrived when a retransmission timeout expires, the request is issued again, up to a configurable maximum number of retransmissions; the timeout is fixed and configurable. Each attempt is forwarded independently and can therefore be served by a different node; in particular, a retransmission may be served by a cache that was populated by an earlier attempt whose data was lost on a later hop. If the round-trip time of an attempt exceeds the timeout, the receiver retransmits before the data arrives; such spurious retransmissions contribute to the network traffic but not to the latency of the chunk.



**Figure 3:** Illustrative chunk sessions. A content is delivered only when all its chunks have been received. (a) The chunk is found in the cache of the serving satellite. (b) No cache on the path holds the chunk, which is served by the origin server and cached on the way back according to the caching strategy. (c) The data is lost on an inter-satellite link; after the timeout the request is retransmitted and served again by the cache. (d) The session starts during the blackout that follows a handover, when the terminal is already associated with its new serving satellite: requests are lost on the user-radio link until the blackout ends, and the retransmission then reaches the new serving satellite.

### 7.3 Timing

Each hop contributes its propagation delay and the transmission (serialisation) time of the packet on that link, which depends on the packet size and on the capacity of the link type. The latency of a delivered chunk is the time elapsed from the first request to the arrival of the data, including the timeouts of the failed attempts; the delivery time of a content is the time elapsed from its first chunk request to the arrival of its last chunk.

## 8 Result collectors

ICARUS-LEO adds three collectors to those of ICARUS (Table 2).

The **chunk-level collector** quantifies caching effectiveness and delivery reliability. The chunk hit ratio is the fraction of chunk requests that find the chunk in an on-path cache; it is computed over all chunk requests, including those that are eventually not delivered. The content-level metrics are computed per content request: the fraction of requests whose chunks were all delivered from caches (full cache hit), the fraction whose chunks were all delivered (full delivery), the average fraction of delivered chunks (average completion) and the statistics of the content delivery time. A catalogue-level variant of the full delivery rate averages over distinct contents rather than over requests.

The **LEO collector** measures the behaviour of the satellite network: traffic broken down by link type, backbone overhead with respect to the user-radio traffic, delivery rate of the sessions affected and not affected by a handover, path length in hops, load balance across the inter-satellite links, and end-to-end

latency with its serialisation component.

The **session log** records, for each session, the sequence of hops with their timing, cache hits, losses and retransmissions, together with the handover events that overlap the session. It is used to generate the interactive session viewers provided as examples in the repository, which display each session as a sequence diagram.

The original ICARUS collectors remain available. Since they were designed for lossless, object-level delivery, their definitions do not account for requests that are lost before reaching a cache or an origin server; for studies with losses, the dedicated collectors should be used.

**Table 2:** Result collectors introduced by ICARUS-LEO and their metric groups.

| Metric group                 | Content                                                                                           |
|------------------------------|---------------------------------------------------------------------------------------------------|
| <i>Chunk-level collector</i> |                                                                                                   |
| CHUNK_HIT_RATIO              | fraction of chunk requests found in an on-path cache                                              |
| CONTENT_COMPLETION           | full cache hit, full delivery, average completion, catalogue full delivery, content delivery time |
| <i>LEO collector</i>         |                                                                                                   |
| LINK_LOAD_LEO                | bytes carried per link type (requests, data, per session)                                         |
| BACKBONE_OVERHEAD            | backbone (ISL and GSL) versus user-radio traffic                                                  |
| HANDOVER_STATS               | delivery rate with and without handover, handover penalty                                         |
| PATH_HOP_COUNT               | hops of requests and data, overall and per link type                                              |
| ISL_LOAD_DISTRIBUTION        | load across inter-satellite links, Jain's fairness index                                          |
| E2E_LATENCY                  | chunk latency over all and over delivered sessions, serialisation delay                           |
| <i>Session log</i>           |                                                                                                   |
| per-session record           | hops, timing, cache hits, losses, retransmissions, handovers                                      |

## 9 Modelling assumptions

ICARUS-LEO inherits the flow-level design of ICARUS and refines it to the level of chunks. This choice keeps simulation campaigns with tens of thousands of requests and wide parameter sweeps affordable, at the price of the following simplifications:

- **Forwarding plane.** Requests follow shortest paths; the stateful forwarding plane of ICN architectures (pending interest tables, request aggregation, adaptive or multipath forwarding) is not modelled.
- **Link contention.** The transmission time of every packet on every hop is modelled, but queueing due to contention among concurrent sessions on the same link is not: each session sees the links as available.
- **Channel.** Losses are described by bit-error rates, Markov channel states or recorded traces, rather than derived from a physical link budget (path loss, fading, antenna patterns).
- **Orbits and dynamics.** Satellites follow ideal circular orbits without perturbations; the propagation delays of the satellite links follow from the geometric distances. In dynamic topologies, changes of the inter-satellite and ground-station links are applied at the granularity of the topology snapshots, whereas the handover instants of the user terminals in the analytical model are not tied to the snapshots. Each session uses the topology in force at its start for all its attempts.
- **Workload.** Content popularity is stationary and follows a Zipf law; all the chunks of a content are requested together.

## References

- [1] L. Saino, I. Psaras, and G. Pavlou, “Icarus: a caching simulator for Information Centric Networking (ICN),” in *Proc. SIMUTOOLS*, 2014. Source code: <https://github.com/icarus-sim/icarus>.
- [2] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, “Web caching and Zipf-like distributions: evidence and implications,” in *Proc. IEEE INFOCOM*, 1999.
- [3] N. Laoutaris, H. Che, and I. Stavrakakis, “The LCD interconnection of LRU caches and its analysis,” *Performance Evaluation*, vol. 63, no. 7, pp. 609–634, 2006.
- [4] I. Psaras, W. K. Chai, and G. Pavlou, “In-network cache management and resource allocation for information-centric networks,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 11, pp. 2920–2931, 2014.
- [5] W. K. Chai, D. He, I. Psaras, and G. Pavlou, “Cache ‘less for more’ in information-centric networks,” in *Proc. IFIP NETWORKING*, 2012.
- [6] J. G. Walker, “Satellite constellations,” *J. British Interplanetary Society*, vol. 37, pp. 559–572, 1984.
- [7] E. N. Gilbert, “Capacity of a burst-noise channel,” *Bell Syst. Tech. J.*, vol. 39, no. 5, pp. 1253–1265, 1960.
- [8] E. O. Elliott, “Estimates of error rates for codes on burst-noise channels,” *Bell Syst. Tech. J.*, vol. 42, no. 5, pp. 1977–1997, 1963.